

Sakthy re-learns Python as Senior

December 4, 2025

Contents

1	Motivation	2
2	Modules and Packages	2
2.1	What's in <code>__init__.py</code> ?	2
2.2	How does package gets imported?	2
2.3	Absolute vs Relative imports	3
3	Concurrency & Asynchronous Programming	3
3.1	Concurrency vs Parallelism	3

1 Motivation

I have learned Python before, but as time has passed, I find myself needing a refresher to get back up to speed with the latest features and best practices. People say there are many ways Python can be written. But I always felt there is one true way to write Python depending on the use case. So this document serves as my internal monologue as I make these decisions throughout my workflow. It contains code snippets, explanations and occasional ramblings about why I choose to do things a certain way. Written by me for me.

2 Modules and Packages

2.1 What's in `__init__.py`?

`__init__.py` is used to mark a directory as a Python package. There seems to be a lot of confusion about what should go inside this file. Some say it needs to be empty and some say we can put some there. But for me (*after lots of reading*), it felt like `__init__.py` should not be empty but should not contain too much code either.

It can contain the following:

- **Package Initialization:** Basically things you want to run when the package is imported. This can include logging, configuration settings, etc.
- **Act as public API:** You can define what is accessible when the package is imported using `__all__` list. So someone who is using this package can know what is the public interface rather than digging into the package internals.
- **Submodule Imports:** This is important for backward compatibility. If you have submodules in your package and you want to make them accessible directly from the package, you can import them in `__init__.py`. This way, users can access submodules without needing to know the internal structure of the package.

But it should **NOT** contain:

- **Heavy Business Logic:** I don't think we should put heavy business logic or complex functions in `__init__.py`. This should be purely an interface file rather than a place to put all the code.
- **Large Imports:** Avoid importing large libraries or modules in `__init__.py` as it can slow down the package import time. Only import what is necessary for the package initialization and what you want to expose as part of the public API.
- **Circular Imports:** Be cautious about importing modules that might lead to circular dependencies. This can cause import errors and make the code harder to maintain.

2.2 How does package gets imported?

When you import a package in Python, the interpreter follows a specific sequence of steps to locate and load the package. Here's a simplified explanation of how it works:

1. **Search in `sys.modules`:** First, Python checks if the package is already loaded in memory by looking in the `sys.modules` dictionary. If it finds the package there, it uses the existing module object.

2. **Locate the Package:** If the package is not found in `sys.modules`, Python searches for the package in the directories listed in `sys.path`. This list includes the current directory, standard library directories, and any directories specified in the `PYTHONPATH` environment variable.
3. **Load the Package:** Once Python locates the package directory, it looks for the `__init__.py` file inside that directory. This file is executed to initialize the package.
4. **Create Module Object:** Python creates a new module object for the package and adds it to `sys.modules`.
5. **Execute `__init__.py`:** The code inside `__init__.py` is executed. This can include setting up package-level variables, importing submodules, and defining the package's public API.
6. **Make the Package Available:** After executing `__init__.py`, the package is now available for use in the importing module. You can access its submodules and attributes as defined in `__init__.py`.

So many steps just to import a package! *So remember, don't ignore the linter error of unused imports lol. The app is dying from all these unnecessary imports slowing down the import time.*

2.3 Absolute vs Relative imports

Absolute imports is just specifying the full path of the module you want to import starting from the project's root directory. For example:

```
1 from my_package.submodule import my_function
```

Listing 1: Absolute Import Example

Relative imports, on the other hand, use dot notation to specify the location of the module relative to the current module. For example:

```
1 from .submodule import my_function # Single dot means current package
2 from ..another_module import another_function # Double dot means parent
   package
```

Listing 2: Relative Import Example

I see more value in absolute module because it remove cognitive load. It's all just there. You know exactly where the module is located in the project structure. There is no need for any mind calculations. But it's not bells and whistles. Relative imports to certain extent decouple the module from the project structure. So if you are moving modules around a lot, relative imports can be useful. But I think absolute imports are the way to go for most cases.

3 Concurrency & Asynchronous Programming

3.1 Concurrency vs Parallelism